

Assignment 1: Data Collection and Preprocessing for Foundation Model Pre-Training

Report

1. Dataset Sources and Total Size

The pipeline collects text from three public Hugging Face datasets, chosen to satisfy the assignment's diversity requirement by spanning three distinct domains:

Domain	Dataset	Docs kept	Notes
Encyclopedic	Salesforce/wikitext (wikitext-103-raw-v1)	440,976	Full English Wikipedia articles, raw text.
News	fancyzhx/ag_news	9,427	Short news articles across four topic categories.
General web	Skylion007/openwebtext	72,087	Reddit-outbound pages with high upvote scores; the open replication of GPT-2's WebText corpus.

All three sources were streamed (streaming=True in the Hugging Face datasets API) rather than fully downloaded, and documents were pulled in round-robin fashion until roughly one-third of the 1 GB target had been collected from each domain. The actual submission run collected **965,168 raw documents totaling 744.21 MB** before cleaning. Encyclopedic text (Wikipedia) reached its share fastest and ended up dominating the corpus (440,976 of the 522,490 final documents, 84.4%), while the news source (ag_news) is a comparatively small, finite dataset and was fully exhausted well before reaching its one-third target, leaving news under-represented at only 9,427 documents (1.8%). Web text (openwebtext) landed in between at 72,087 documents (13.8%). This observed imbalance, and how to address it, is discussed further in Section 5.

Pipeline validation before the full run. The development environment used to build this pipeline initially had no internet access, so an auto-detected offline mode (--demo) was used first, on a small self-authored synthetic corpus, to validate the cleaning, tokenization, and DataLoader logic end-to-end before pointing the script at the real, internet-connected collection path described above. The --demo flag remains in the script for future offline debugging, but every statistic in the remainder of this report comes from the real, full-scale run (python data_collection_preprocessing.py --target-gb 1.0), which completed in 349.87 seconds (~5.8 minutes) using the genuine GPT-2 tokenizer and PyTorch backend.

2. Cleaning Strategies and Reasoning

The cleaning stage applies the following steps, in order, to every raw document:

- **HTML-entity unescaping and tag stripping** — web-sourced text (OpenWebText, and occasionally Wikipedia markup) contains raw HTML entities (&) and tags; both are removed so the tokenizer never wastes vocabulary slots on markup.
- **Markdown normalization** — markdown links are collapsed to their visible text ([label](url) → label) rather than deleted outright, since the link text is often meaningful prose; residual emphasis characters (*, _, #, backticks) are stripped.
- **Wikipedia reference-marker removal** — patterns like [12] or [citation needed] are removed, since they are typographic noise rather than semantic content.
- **Whitespace normalization** — repeated spaces, tabs, and newlines are collapsed to single spaces so that near-identical documents differing only in formatting hash to the same value during deduplication.

- **Exact-duplicate removal** — each cleaned document is hashed with SHA-256; a document whose hash has already been seen is dropped. Exact-duplicate removal was prioritized over more expensive near-duplicate detection (e.g., MinHash/LSH) given the assignment's scope, and is listed as an optional extension in Section 5.
- **Short-document filtering** — documents under 50 words are dropped, since very short fragments (navigation text, stub pages, one-line comments) contribute noise without enough context for the model to learn meaningful language patterns from them.

Lowercasing trade-off. The assignment brief lists lowercasing as a normalization step, but it is implemented as an opt-in flag (`--lowercase`, default off) rather than always-on. The GPT-2 byte-level BPE tokenizer used for the real run is case-sensitive by design — case carries information (proper nouns, sentence starts, acronyms like "NASA" vs. "nasa") that a foundation model benefits from learning to model rather than discarding. Lowercasing is retained as a flag so it can be enabled for ablation experiments or for simpler downstream tokenizers that assume normalized case.

Actual submission run results. Cleaning and deduplication were applied to all 965,168 raw documents. 442,678 documents (45.9%) were removed — primarily exact duplicates (Wikipedia in particular contains many near-identical stub/redirect pages once whitespace and markup are normalized) and sub-50-word fragments — leaving **522,490 cleaned documents totaling 693.21 MB**. That nearly half of the raw corpus was removed by cleaning is itself a notable finding, discussed further in Section 5.

3. Tokenization Choices

Tokenizer: `transformers.AutoTokenizer.from_pretrained("gpt2")` — GPT-2's byte-level Byte-Pair Encoding (BPE) tokenizer, vocabulary size 50,257. Byte-level BPE was chosen over WordPiece for two reasons: (1) it operates on raw UTF-8 bytes, so it never produces an out-of-vocabulary/<unk> token regardless of what characters appear in web-scraped text (emoji, non-English words, unusual punctuation all encode cleanly), and (2) it is the tokenizer family most directly associated with decoder-only, causal-language-model pretraining, which is the typical target architecture for this kind of corpus. GPT-2 has no dedicated pad token, so `tokenizer.pad_token` is set to `tokenizer.eos_token` for padding purposes.

Block size: 512 tokens (configurable via `--block-size`). This value was chosen as a practical middle ground: large enough to capture multi-sentence context in most documents, but small enough to keep memory and attention compute (which scales quadratically with sequence length in a standard transformer) manageable on modest hardware for a course assignment.

Chunking long documents: Documents whose tokenized length exceeds the block size are split into consecutive chunks of exactly `block_size` tokens; an optional `--stride` parameter creates overlapping chunks so information near a chunk boundary is not seen only once. The final, shorter chunk of a document is kept (not discarded) and padded later by the `DataLoader`'s `collate` function, so chunking never silently drops the tail of a long article.

Actual submission run results. The real GPT-2 `AutoTokenizer` (vocabulary size 50,257) tokenized the 522,490 cleaned documents into **637,919 token blocks** of up to 512 tokens each — more blocks than cleaned documents, confirming that a meaningful share of documents (particularly longer Wikipedia articles) were chunked into multiple blocks rather than fitting in a single 512-token block, as intended. The tokenizer library itself logged an informational notice for documents whose raw token length exceeded GPT-2's original 1,024-token design limit; this is expected and harmless here, since chunking (not GPT-2's built-in limit) is what actually splits long documents in this pipeline.

4. Custom Data Loader Implementation

Two complementary loader implementations are provided, both built on `torch.utils.data`:

- **TokenizedTextDataset(Dataset)** — a standard map-style dataset over an in-memory list of already-tokenized blocks. Used when the tokenized corpus fits comfortably in RAM.
- **StreamingTokenizedDataset(IterableDataset)** — tokenizes documents lazily, one at a time, from a generator, and yields token blocks on the fly. This is the mechanism used to avoid the "memory bottleneck" the assignment calls out: at no point does the full tokenized corpus need to exist in memory simultaneously.

Both feed into a shared `DataLoader` configured with `shuffle=True` and a custom `collate_fn` that (a) finds the longest sequence in each batch, (b) right-pads every shorter sequence in that batch up to that length using the tokenizer's `pad_token` id, and (c) builds a matching binary attention mask so the model does not attend to padding positions. Padding to the batch maximum (rather than a fixed global maximum) minimizes wasted computation on pad tokens.

compared to always padding to block_size.

Dependency-free fallback loader. When torch is not installed, an equivalent FallbackDataLoader class reproduces the same batching, shuffling, and padding behavior using plain Python lists in place of tensors, so the DataLoader logic itself (not just the tokenizer) is verifiable without the torch dependency. This is the path exercised in the validation run below; the real submission run uses the genuine torch-backed classes and calls torch.save() to produce sample_dataset.pt.

Actual submission run results. The 637,919 tokenized blocks were wrapped in the real torch-backed TokenizedTextDataset / DataLoader (confirmed by "torch_backend": "torch" in the run's output statistics) and batched with batch_size=8. The first 8 batches were saved via genuine torch.save() to sample_dataset.pt, which loads back with torch.load() as a list of dicts each containing input_ids and attention_mask tensors of shape [8, max_len_in_batch] and a parallel list of domain labels — confirming correct batch shape, correct padding, and correct end-to-end provenance tracking from raw document through to tokenized, batched tensor.

5. Challenges Encountered

- **Domain imbalance from uneven dataset sizes.** Although the collection logic targets an equal one-third raw-byte share per domain, ag_news is a comparatively small, finite dataset and was fully exhausted before reaching its target share, while Wikipedia (an effectively much larger source) filled its share quickly and continued streaming past what was strictly needed relative to the other two domains before the loop moved on. The result was a corpus skewed heavily toward encyclopedic text (84.4% of final documents) and news badly under-represented (1.8%). This was only visible after running the real collection against the live datasets; the offline synthetic corpus used for pre-run validation had deliberately equal per-domain document counts and could not have surfaced this issue. Section 6 discusses the training-quality implications and proposes a byte-based (rather than document-count-based) rebalancing fix.
- **Development-environment internet restrictions.** The initial practical challenge was building and validating a pipeline meant to stream gigabytes of data from Hugging Face while working in a sandbox with no outbound network access. This was addressed by writing the collection, tokenization, and DataLoader stages behind small interfaces (.encode(), __len__ / __getitem__, batch iteration) that could be backed by either the real libraries or a lightweight offline substitute, so the control flow, chunking arithmetic, and padding logic could be fully exercised and debugged before the real run was executed on internet-connected hardware.
- **Dataset naming changes.** Hugging Face reorganized several community datasets under canonical namespaces during 2024 (for example, the bare name wikitext now needs to be requested as Salesforce/wikitext under current versions of the datasets/huggingface_hub libraries, or a HfUriError is raised). The collection stage tries several known name variants per domain, in order, so the script continues to work regardless of which library version and naming convention is installed.
- **Memory bottlenecks at scale.** A naive implementation that downloads a full dataset, tokenizes every document eagerly, and only then builds a Dataset would require holding the entire tokenized 700 MB+ corpus in memory at once. The streaming collection (datasets streaming=True) plus the optional StreamingTokenizedDataset (an IterableDataset that tokenizes one document at a time) directly address this: neither the raw text nor the tokenized corpus needs to be fully materialized simultaneously.
- **Deduplication cost vs. benefit.** Exact-duplicate removal via hashing is $O(n)$ and cheap, and in the real run it accounted for a substantial share of the 45.9% of documents removed during cleaning — confirming that raw Wikipedia/web-sourced data contains meaningfully more redundant content than the corpus's raw byte count would suggest. It will not, however, catch near-duplicates (e.g., a news wire story republished with a different byline). A near-duplicate detector (MinHash/LSH) would catch more noise but adds meaningful implementation and runtime cost; it is called out as an optional extension in Section 6.
- **Variable-length batching.** Real text documents vary enormously in length, and naively padding every sequence in the dataset to a single global maximum wastes substantial compute on pad tokens. The custom collate_fn instead pads only to the longest sequence within each individual batch, which is a standard mitigation but does require every downstream training step to correctly consume the accompanying attention mask rather than assuming a fixed sequence length.

6. Reflections on Preprocessing's Impact on Pretraining Quality

This assignment reinforced that data preprocessing is not a peripheral engineering step but a direct determinant of what a foundation model can learn. A few concrete takeaways:

- **Deduplication affects effective diversity, not just size.** A corpus that is nominally 1 GB but contains substantial duplicate content is effectively smaller and less diverse than its raw byte count suggests; models trained on heavily duplicated data have been shown in prior work to be more prone to memorizing and regurgitating repeated passages verbatim, which is undesirable for a general-purpose foundation model.
- **Tokenizer choice shapes what "a token" means to the model.** Byte-level BPE guarantees full coverage of the input character set at the cost of sometimes splitting rare words into several sub-word pieces; this is a reasonable trade-off for a broad, multi-domain corpus (encyclopedic + news + informal web text) where vocabulary diversity is high and an out-of-vocabulary token would otherwise be common.
- **Domain balance is itself a preprocessing decision — and easy to get wrong in practice.** Collecting roughly equal raw bytes from encyclopedic, news, and web-forum-style text was a deliberate goal, but the real run's actual domain split (84.4% encyclopedic, 13.8% web, only 1.8% news) shows that intent alone does not guarantee balance: it also depends on how large the underlying source datasets are and in what order they are streamed. The domain label carried alongside each token block throughout the pipeline made this imbalance immediately measurable rather than silently baked in. A model pretrained on this exact corpus as-is would likely over-fit to encyclopedic register and under-learn news style; the practical fix, noted for a future run, is to cap collection per-domain by document count from multiple news sources combined (or continue streaming `ag_news` alongside a second news source such as `cc_news`) rather than relying on a single finite dataset to fill an entire domain's byte quota.
- **Length filtering trades a small amount of data for a larger amount of signal.** Discarding sub-50-word documents removes some raw bytes, but those documents (navigation stubs, one-line comments) carry very little of the long-range linguistic structure that pretraining is trying to teach the model in the first place, so the trade is favorable.
- **Efficient batching is a training-quality concern, not just a speed concern.** Padding strategy and streaming affect not only wall-clock time but also how much of each training step is spent on real signal versus pad tokens, which in turn affects how many real tokens a fixed compute budget can expose the model to during pretraining.

Appendix: Reproducing This Run

The results reported throughout this document were produced with, in an internet-connected environment with `pip install datasets transformers torch tqdm`:

```
python data_collection_preprocessing.py --target-gb 1.0 --tokenizer gpt2 --block-size 512
--batch-size 8 --out-dir ./output
```

This streamed the three domains listed in Section 1, applied the cleaning and tokenization pipeline described above using the real GPT-2 tokenizer, and wrote `output/sample_dataset.pt` (genuine `torch.save()` tensors, submitted alongside this report) and `output/run_stats.json` (the exact statistics reported throughout this document) into `--out-dir`. Total wall-clock time was 349.87 seconds.

An offline validation mode used during development can be reproduced with: `python data_collection_preprocessing.py --demo` (no internet or extra packages required); it exercises the same code paths on a small synthetic corpus and is not a substitute for the run above.